

Applied Programming For AI

GIT - version control

Check in

- Introductions
- Does everyone have a group?
- GitHub usernames

Agenda

- Questions:
 - Please ask them as we go
- Git - Some Background
- Demonstration Basic Commands
- Setup of Your Project
- Common Pitfalls
- Free Experimentation
- Discussion Projects

Background

- What is Git?
 - Definition: Git is a distributed version control system.
 - Used for: Tracking code changes, collaboration, backup.
- Why Use Git?
 - Never lose progress
 - Collaborate safely
 - Experiment without risk

Why Do We Care?

- Industry Standard
 - Important skill in software engineering
 - Open source code is often shared through GitHub
- A Journal of Your Work
 - Cooperation and progress become clear through your git history
 - Part of your grade is determined by your contributions

Key Concepts

- Repositories
- Commits
- Branches
- Merges

Repository

- A project folder managed by Git
- Tracks all changes to files over time
- Contains the entire history of the project
- Can be local (on your machine) or remote (on platforms like GitHub)
 - So Git $\neq$ GitHub
 - Every team member will have a local repository of their own and one shared remote repository

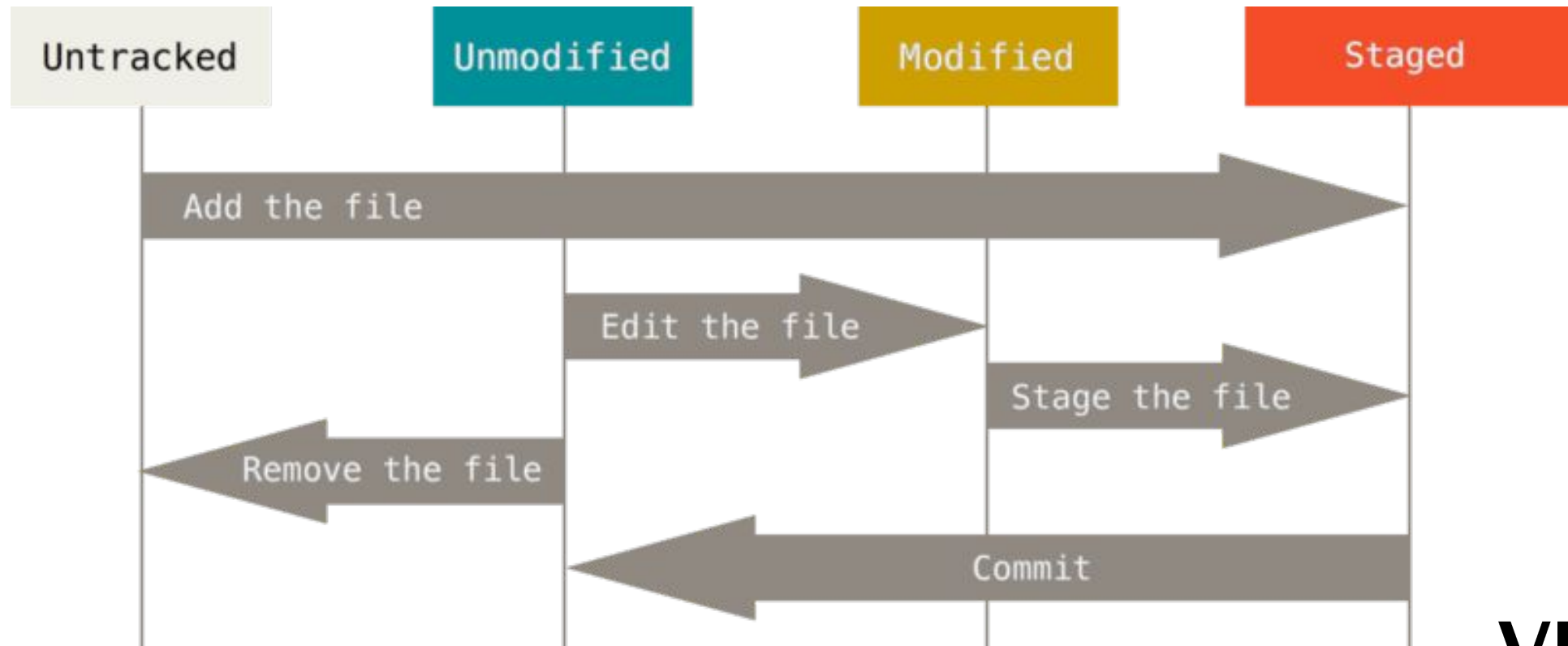
Commits

- A snapshot of your files at a specific point in time
- A way to save your progress with a message describing the change
- Recorded history of what changed, who changed it, and when

"A save point in a video game — you can always go back to it."

Commits - state of files

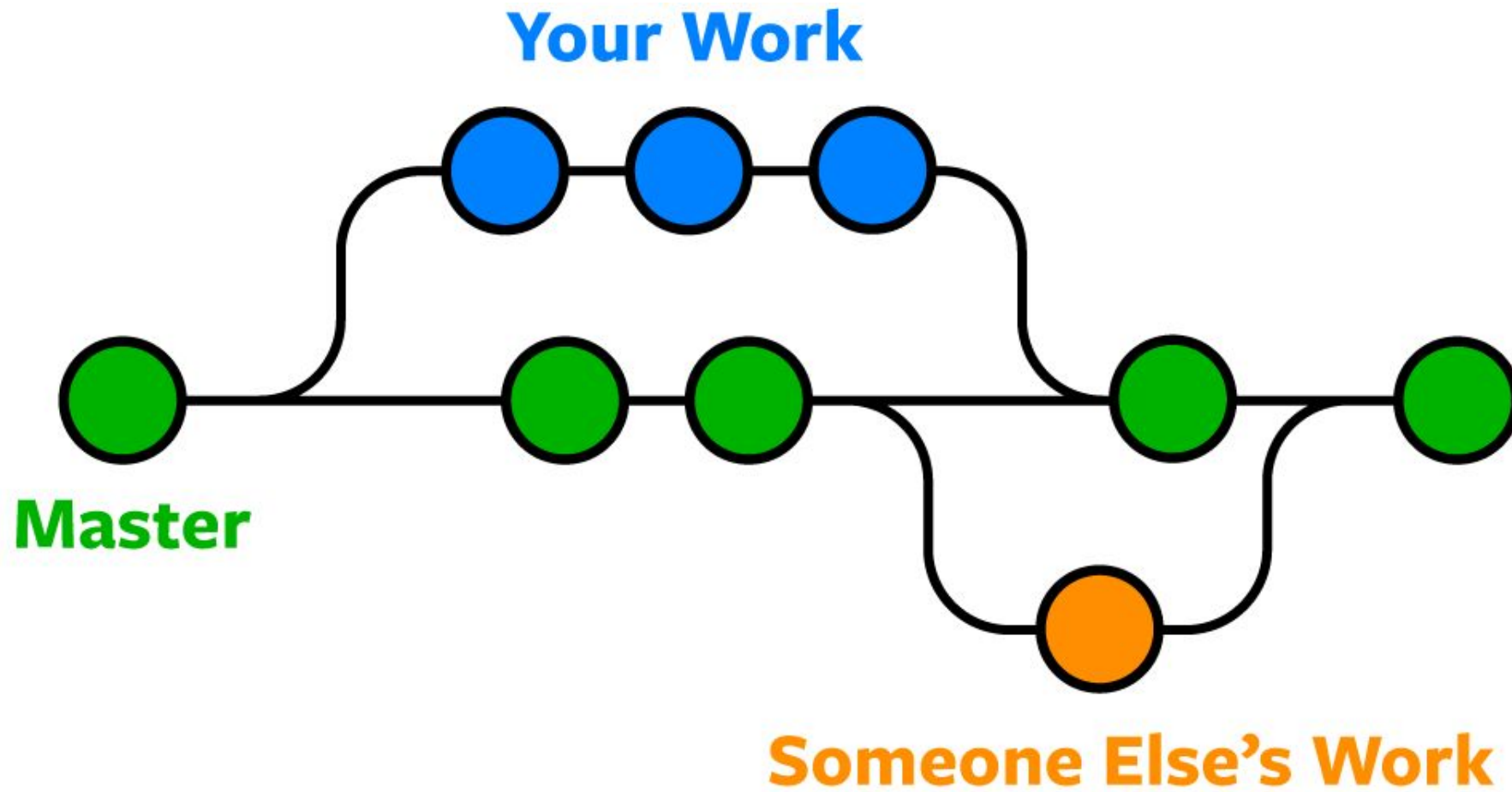
Scary graph, we will get back to it later



Branches

- A parallel version of your codebase
- Lets you work on features, fixes, or experiments without changing the main code
- Can be merged back into the main line when ready
- You will do most of your work in your personal branches
 - Each repository can have its own branches
 - You will need personal branches on the remote repository

Branches



Merging

- The process of combining changes from one branch into another
 - While collaborating this is often your local version of a branch with the remote version
- Typically done when a feature is ready to be added back to the main branch
- Git tries to auto-resolve differences, but sometimes you'll need to help
 - We will show some examples later

Basic Commands

Follow along from here

Creating a Repository

First things first if you have just installed git:

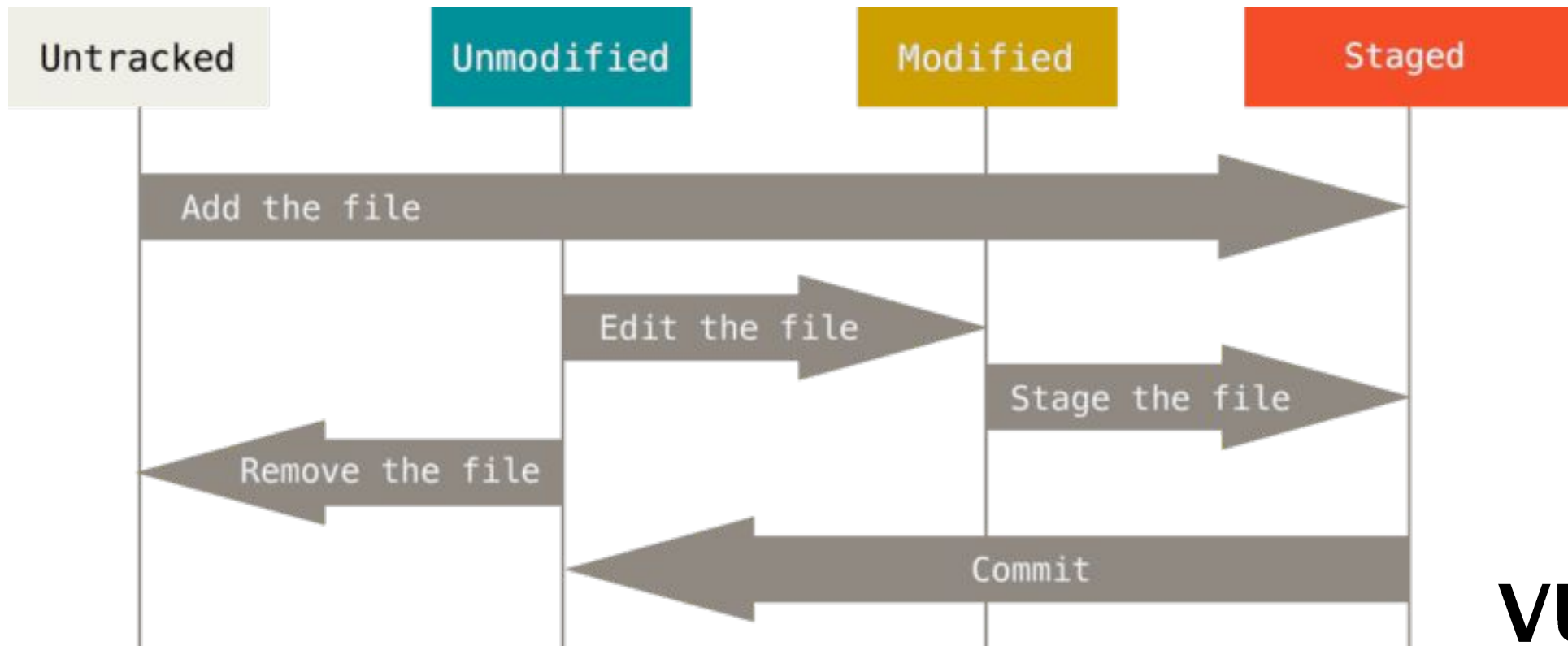
- `git config --global user.name "Your Name"`
- `git config --global user.email "Your Email"`

- `git init` - Turn the a folder into a git tracked repository

- `git clone [url]` - Copy a remote repository

Adding and Committing

- `git add [files]` - stages new files or modified files
- `git commit` - creates a new commit
 - Commits always need a message! (`git commit -m "some message"`)



Pushing and Pulling

- `git push [remote] [branch]` -
update the remote branch with your local commits
- `git fetch [remote]` - Download the state of the remote
 - This does not make any changes to your files
- `git pull [remote] [branch]` -
Merge the remote branch into your current branch

If you cloned your repository the remote repository is called **origin**

Example Merge Conflict

Sometimes git needs your help

Branches

- `git branch [name]` - Create a branch with a certain name
- `git switch [name]` - Switch to work in a certain branch
 - `git switch -c [name]` creates and switches to the branch
- `git merge [branch]` - Merge the target branch into the current
 - This is also subject to merge conflicts
- ! Remote branches and your local branches are not the same !

Your Project

- Remote repository hosted on GitHub
- main branch that will always contain a working version of the project
- dev branch for the team to collaborate on releases to main
- Personal remote branches for each team member
 - Push your work daily to your personal branch
 - You can have multiple personal branches for each feature
- Every project needs a README.md
 - Besides an introduction to the project, it should also contain installation instructions
 - Also a requirements.txt

Pitfalls

- Working together on the same computer
 - This is fine but should be added to the commit messages
 - Make sure you still make some personal contributions
- Meaningless commit messages
 - Try to think of which feature or bug you were working on
 - Do not commit everything in one go but try to make different ones
- Committing to the wrong branch
 - For example introducing experimental code to the main branch
 - If you are still local you can reset
 - If you pushed to remote then make a new commit to revert the changes

Free Experimentation

- Make sure to set up your project
- But also try out all the commands you learned today
- Feel free to ask any questions
- We will also come to discuss your projects